



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

存储器管理模拟实验

专业班级:

计算机 1902 班

学 号:

1906410229

姓 名:

蔡艺洵

日 期:

2021 年 11 月 10 号

【实验目的】

1. 通过编写和调试存储管理的模拟程序以加深对存储管理方案的理解；
2. 熟悉虚存管理的各种页面淘汰算法；
3. 通过编写和调试地址转换过程的模拟程序以加强对地址转换过程的了解。

【实验准备】

1. 虚拟存储器的管理方式
 - 段式管理
 - 页式管理
 - 段页式管理
2. 页面置换算法
 - 先进先出置换算法
 - 最近最久未使用置换算法
 - Clock 置换算法
 - 其他置换算法

【实验内容】

1. 实验题目

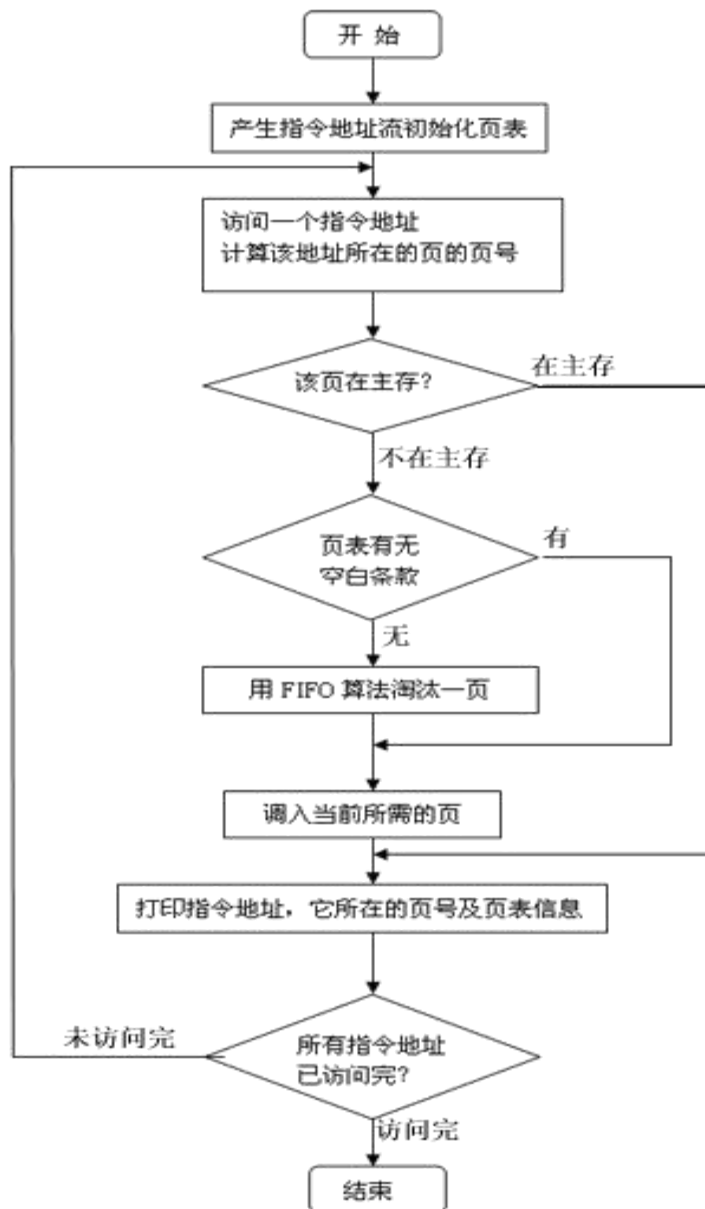
设计一个请求页式存储管理方案。并编写模拟程序实现之。产生一个需要访问的指令地址流。它是一系列需要访问的指令的地址。为不失一般性，你可以适当地（用人工指定地方法或用随机数产生器）生成这个序列，使得 50% 的指令是顺序执行的。25% 的指令均匀地散布在前地址部分，25% 的地址是均匀地散布在后地址部分。为简单起见。页面淘汰算法采用 FIFO 页面淘汰算法，并且在淘汰一页时，只将该页在页表中抹去。而不再判断它是否被改写过，也不将它写回到辅存。

2. 具体做法

产生一个需要访问的指令地址流；指令合适的页面尺寸（例如

以 1K 或 2K 为 1 页)；指定内存页表的最大长度，并对页表进行初始化；每访问一个地址时，首先要计算该地址所在的页的页号，然后查页表，判断该页是否在主存——如果该页已在主存，则打印页表情况；如果该页不在主存且页表未满，则调入一页并打印页表情况；如果该页不在主存且页表已满，则按 FIFO 页面淘汰算法淘汰一页后调入所需的页，打印页表情况；逐个地址访问，直到所有地址访问完毕。

3. 存储管理算法的流程图



【源程序代码】

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <iomanip>
#include <string>
#include <ctime>
#include <windows.h>
#include <fstream>

using namespace std;
//以下是页面信息类的定义
class PageInformation
{
protected:
    int pid;           //页号
    int count;         //统计驻留在物理块中的次数（时间），初始为0
public:
    PageInformation() { count = 0; }           //默认构造函数，驻留在页面的
    时间初始化为0
    PageInformation(PageInformation &p)
    {
        p.pid = pid;
        p.count = count;
    }
    virtual ~PageInformation() {}
    void setpid(PageInformation *headptrblock, int pid)//修改物理块对象数组成员
    pid
    {
        headptrblock->pid = pid;
    }
    void setpidrand(PageInformation *headptr, int pid)//修改页面对象数组成员
    pid
    {
        headptr->pid = pid;cout << pid << " ";
    }
    void display()
    {
        cout << pid << " ";
    }
};
```

```

        //cout<<pid;
    }
    void setpid()//选择从键盘输入时设置对象数组成员 pid
    {
        int pidtemp;
        cout << "请输入您的页号 (范围 1-9): ";
        cin >> pidtemp;
        this->pid = pidtemp;
    }
    void counter()
    {
        count++;
    }
    void resetcount()
    {
        count = 0;
    }
    int getpid()
    {
        return pid;
    }
    int getcount()
    {
        return count;
    }
}; //PageInformation

void Directloading(PageInformation *&headptr, PageInformation *&headptrblock,
int pagenum)//直接将前三个页面装入物理块中
{
    int temp;
    for (int i = 0; i < pagenum; i++)//直接将符合物理块数大小的页面装入物理块中
    {
        (headptrblock + i)->setpid((headptrblock + i), (headptr +
i)->getpid());
        temp = i + 1;
        for (int j = 0; j < temp; j++)
            (headptrblock + j)->counter(); //对应物理块驻留时间++
    }
    cout << "原始各物理块页号为: " << "\t\t";
}

```

```

    for (int i = 0; i < pagenum; i++)
    {
        (headptrblock + i)->display();
    }
    cout << endl;
} //Directloading

void OPT(PageInformation *headptr, PageInformation *headptrblock, int pagenum,
int pagenumber, int &pagecounter)
{
    pagecounter = 0; //置 0
    Directloading(headptr, headptrblock, pagenum);
    int aid;
    int status = 1;
    int counter[3];
    aid = 3;
    int temp = 0;
    for (int i = pagenum; i < pagenumber; i++) //如果有缺页现象发生，就将存入物
    理块时间最久的一个页面取出，装入新的页面
    {
        for (int j = 0; j < pagenum; j++)
        {
            (headptrblock + j)->counter(); //对应物理块驻留时间++

        }
        for (int j = 0; j < pagenum; j++) //无缺页跳出循环进行下一页面判断，缺页
        寻找未来最久未使用的页面置换之
        {
            if ((headptrblock + j)->getpid() == (headptr + i)->getpid())
            {
                status = 0;
                pagecounter++;
            }
        }
        for (int j = 0; j < pagenum; j++)
        {
            for (int k = i; k < pagenumber; k++)
            {
                if ((headptrblock + j)->getpid() == (headptr + k)->getpid())
                {

```

```

        counter[j] = k;
        break;
    }
    else
        counter[j] = 21;
    }
}
for (int j = 0; j < pagenum; j++)
{
    if (counter[j] > aid)
    {
        aid = counter[j];
        temp = j;
    }
}
if (status)
{
    (headptrblock + temp)->setpid((headptrblock + temp), (headptr +
i)->getpid()); //将新页面装入物理块置换掉 temp 页面
    (headptrblock + temp)->resetcount();
}
cout << "第" << setw(2) << i + 1 << "新页面载入时各物理块页号为: ";
for (int j = 0; j < pagenum; j++)
{
    (headptrblock + j)->display();
}
if (status)
{
    cout << "! 发生缺页! ";
}
cout << endl;
//将临时暂存变量更新
temp = 0;
aid = 3;
status = 1;
}
int pagecounterrate = ((17 - pagecounter) * 100 / 20);
cout << "使用最佳适应算法的缺页数为: " << 17 - pagecounter << "缺页率为:
" << "% " << pagecounterrate << endl;

```

```

} //opt

void FIFO(PageInformation *headptr, PageInformation *headptrblock, int pagenum,
int pagenumber, int &pagecounter)
{
    Directloading(headptr, headptrblock, pagenum);
    pagecounter = 0; //置0
    int status = 1;
    int temp = 0;
    for (int i = pagenum; i < pagenumber; i++) //如果有缺页现象发生，就将存入物
    理块时间最久的一个页面取出，装入新的页面
    {
        for (int j = 0; j < pagenum; j++)
        {
            (headptrblock + j) -> counter(); //对应物理块驻留时间++

        }
        for (int j = 0; j < pagenum; j++) //无缺页跳出循环进行下一页面判断，缺页
        寻找驻留时间最长的页面置换之
        {
            if ((headptrblock + j) -> getpid() == (headptr + i) -> getpid())
            {
                status = 0;
                pagecounter++;
            }
        }
        for (int j = 0; j < pagenum; j++)
        {
            if ((headptrblock + j) -> getcount() > (headptrblock +
temp) -> getcount())
                temp = j;
        }
        if (status)
        {
            (headptrblock + temp) -> setpid((headptrblock + temp), (headptr +
i) -> getpid()); //将新页面装入物理块置换掉 temp 页面
            (headptrblock + temp) -> resetcount();
        }
        cout << "第" << setw(2) << i + 1 << "新页面载入时各物理块页号为：";
        for (int j = 0; j < pagenum; j++)

```



```

    {
        (headptrblock + j)->display();
    }
    if (status)
    {
        cout << "！发生缺页！";
    }
    cout << endl;
    //将临时暂存变量更新
    temp = 0;
    status = 1;
}

int pagecounterrate = ((17 - pagecounter) * 100 / 20);
cout << "使用最佳适应算法的缺页数为：" << 17 - pagecounter << "缺页率为："
<< "\%" << pagecounterrate << endl;
} //fifo

void LRU(PageInformation *headptr, PageInformation *headptrblock, int pagenum,
int pagenumber, int &pagecounter)
{
    Directloading(headptr, headptrblock, pagenum);
    pagecounter = 0;
    int status = 1;
    int temp = 0;
    for (int i = pagenum; i < pagenumber; i++) //如果有缺页现象发生，就将存入物
    理块时间最久的一个页面取出，装入新的页面
    {
        for (int j = 0; j < pagenum; j++)
        {
            (headptrblock + j)->counter(); //对应物理块驻留时间++
        }
        for (int j = 0; j < pagenum; j++) //无缺页跳出循环进行下一页面判断，缺页
        寻找最未使用的页面置换之
        {
            if ((headptrblock + j)->getpid() == (headptr + i)->getpid())
            {
                (headptrblock + j)->resetcount();
                status = 0;
                pagecounter++;
            }
        }
    }
}

```

```

        }
    }
    for (int j = 0; j < pagenum; j++)
    {
        if ((headptrblock + j)->getcount() > (headptrblock +
temp)->getcount())
            temp = j;
    }
    if (status)
    {
        (headptrblock + temp)->setpid((headptrblock + temp), (headptr +
i)->getpid()); //将新页面装入物理块置换掉 temp 页面
        (headptrblock + temp)->resetcount();
    }
    cout << "第" << setw(2) << i + 1 << "新页面载入时各物理块页号为: ";
    for (int j = 0; j < pagenum; j++)
    {
        (headptrblock + j)->display();
    }
    if (status)
    {
        cout << "! 发生缺页! ";
    }
    cout << endl;
    //将临时暂存变量更新
    temp = 0;
    status = 1;
}
int pagecounterrate = ((17 - pagecounter) * 100 / 20);
cout << "使用最佳适应算法的缺页数为: " << 17 - pagecounter << "缺页率为:
" << "\%" << pagecounterrate << endl;

} //lru

int pagenum = 3; //页框数-物理块数
int pagenumber = 20; //页面数量
int pagecounter = 0;

//以下是主函数
int main(int argc, char **argv)

```

```

{
    time_t Time;
    time(&Time);
    int control;
    PageInformation *headptr = new PageInformation[pagenum];    //生成页面
序列
    PageInformation *headptrblock = new PageInformation[pagenum]; //生成页框
    if (argc == 1)//是否向 main 函数传入参数
    {
        cout << "当前系统时间是:" << ctime(&Time);
        cout << "选择从键盘读入数据扣 1，默认选择随机生成数据" << endl;
        int status;
        cout << "请输入你的选择： ";
        cin >> status;
        if (status == 1)
        {
            cout << "（从键盘中读入数据）" << endl;
            for (int i = 0; i < pagenum; i++)
            {
                (headptr + i)->setpid();
            }
        }
        else
        {
            cout << "（随机生成数据）" << endl;
            srand((unsigned)time(&Time));//srand((unsigned) time (NULL));也
可以传入空指针
            cout << "随机生成页面序列为： ";
            for (int i = 0; i < pagenum; i++)
            {
                (headptr + i)->setpidrand((headptr + i), (int)rand() % 10);
            }
            cout << endl;
        }
    }
    else//从文件中读入数据
    {
        cout << "（文件读入数据）" << endl;
        ifstream in(argv[1]);
        if (in.is_open())

```

```

    {
        cout << "文件打开成功！" << endl;
    }
    else
    {
        cout << "文件打开失败！";
        exit(0);
    }
    cout << "所读取的页面序列为：";
    int j = 0;
    for (int p; in >> p; )
    {
        (headptr + j)->setpidrand((headptr + j), p); //将文件数据读入
        j++;
    }
    in.close();
    cout << endl;
}
do
{
    cout << "当前系统时间是：" << ctime(&Time) << endl;
    cout << " |=====页面置换算法管理系统
=====|" << endl
        << " |                                *1. OPT 最佳置换算法
|" << endl
        << " |                                *2. FIFO 先进先出置换算法
|" << endl
        << " |                                *3. LRU 最近最久未使用置换算法
|" << endl
        << " |                                *0. 退出
|" << endl
        << endl;
    cout << "请输入你的指令：";
    cin >> control;
    switch (control)
    {
    case 1:
        system("cls");
        cout << " (OPT 最佳置换算法) " << endl;
        OPT(headptr, headptrblock, pagenum, pagenumber, pagecounter);

```

```

        break;
    case 2:
        system("cls");
        cout << " (FIFO 先进先出置换算法) " << endl;
        FIFO(headptr, headptrblock, pagenum, pagenumber, pagecounter);
        break;
    case 3:
        system("cls");
        cout << " (LRU 最近最久未使用置换算法) " << endl;
        LRU(headptr, headptrblock, pagenum, pagenumber, pagecounter);
        break;
    case 0:
        cout << "                                感谢使用！ 3 秒后程序结束" <<
endl;

        Sleep(3000);
        break;
    default:
        cout << "                                输入错误！请重新输入：" <<
endl;
        Sleep(1000);
        system("cls");
    }
} while (control != 0);
delete[] headptr;
return 0;
}

```

运行截图：

```
C:\Users\蔡某某\source\repos\operatingsystem3\Debug\operatingsystem3.exe
(FIFO先进先出置换算法)
原始各物理块页号为: 3 4 2
第 4 新页面载入时各物理块页号为: 1 4 2 ! 发生缺页!
第 5 新页面载入时各物理块页号为: 1 5 2 ! 发生缺页!
第 6 新页面载入时各物理块页号为: 1 5 3 ! 发生缺页!
第 7 新页面载入时各物理块页号为: 9 5 3 ! 发生缺页!
第 8 新页面载入时各物理块页号为: 9 1 3 ! 发生缺页!
第 9 新页面载入时各物理块页号为: 9 1 3
第10新页面载入时各物理块页号为: 9 1 2 ! 发生缺页!
第11新页面载入时各物理块页号为: 9 1 2
第12新页面载入时各物理块页号为: 9 1 2
第13新页面载入时各物理块页号为: 9 1 2
第14新页面载入时各物理块页号为: 9 1 2
第15新页面载入时各物理块页号为: 8 1 2 ! 发生缺页!
第16新页面载入时各物理块页号为: 8 0 2 ! 发生缺页!
第17新页面载入时各物理块页号为: 8 0 1 ! 发生缺页!
第18新页面载入时各物理块页号为: 8 0 1
第19新页面载入时各物理块页号为: 6 0 1 ! 发生缺页!
第20新页面载入时各物理块页号为: 6 2 1 ! 发生缺页!
使用最佳适应算法的缺页数: 11缺页率为: %55
当前系统时间是:Tue Dec 14 21:26:17 2021

=====页面置换算法管理系统=====
*1. OPT最佳置换算法
*2. FIFO先进先出置换算法
*3. LRU最近最久未使用置换算法
*0. 退出

请输入你的指令: _
```

【实验体会】

通过本次的操作系统上机实验，我学习了关于先进先出置换算法、最近最久未使用置换算法、Clock 置换算法等算法，在学习这些算法之后，尝试着用代码去实现这些算法，在实现这些页面置换算法的过程中去体会这些算法的优劣，在实践的过程中加深我对这些算法的理解。

【思考题】

1. 几种页面置换算法各有什么优缺点？

FIFO 页面置换算法易于理解和编程。然而，它的性能并不总是十分理想。一方面，所置换的页面可以是很久以前使用过但现已不再使用的初始化模块。另一方面，所置换的页面可以包含一个被大量使用的变量，它早就初始化了，但仍在不断使用。对于局部性强的程序会出现 Belady 异常。

OPT 最好的情况就是 OPT。然而难以实现，谁知道未来要用哪个，一般用作研究其他算法时作为邻居家的孩子比较。

LRU 比较接近 OPT，但是需要硬件支持。